produces:

```
Ho! Ho! Ho!
```

It's easy to avoid fence-post and off-by-one errors; this loop will execute three times, period. In addition to times, integers can loop over specific ranges by calling downto and upto, and all numbers can loop using step. For instance, a simple "for" loop that runs from 0 to 9 (something that you'd write in JavaScript as for(let i = 0; i < 10; i++)) is written as follows:

```
0.upto(9) do |x|
  print x, " "
end
```

produces:

```
0 1 2 3 4 5 6 7 8 9
```

A loop from 0 to 12 by 3 can be written as follows:

```
0.step(12, 3) { |x| print x, " " }
```

produces:

```
0 3 6 9 12
```

Similarly, iterating over arrays and other containers is easy if you use their each method:

```
[ 1, 1, 2, 3, 5 ].each { |val| print val, " " }
```

produces:

```
1 1 2 3 5
```

And once a class supports each, it can also include Enumerable, and the additional methods in the Enumerable module become available. (We talked about this back in the Modules chapter on page 101.) For example, the File class provides an each method, which returns each line of a file in turn. Using the grep method in Enumerable, we could iterate over only those lines that end with a d:

```
File.open("ordinal").grep(/d$/) do |line|
  puts line
end
```

produces:

```
second
third
```

for ... in

Earlier we said that the only built-in Ruby looping primitives were while and until. Technically, that's not true, Ruby does have a for keyword. What's this for thing, then? Well, for is a different way to write an each loop.

When you write this:

```
for song in playlist
  song.play
end
```

Ruby translates it into something like this:

```
playlist.each do |song|
  song.play
```